

Exemplo de derivação sintática:

Objetivo: mostrar como um pequeno trecho de código pode ser construído a partir de regras gramaticais de uma linguagem real.

Fonte de referência: Free Pascal Language Reference Guide, seções "While statement" e "If..then..else statement". A gramática abaixo foi simplificada para fins didáticos, preservando a estrutura sintática essencial dos comandos utilizados.

1. Código gerado

```
program ExemploWhileIF;
var
  i: integer;
begin
  i := 1;
  while i <= 20 do
  begin
    if i mod 2 = 0 then
      writeln(i, 'eh par!')
    else
      writeln(i, 'eh impar!');
    i := i + 1;
  end;
end.
```

2. Regras gramaticais utilizadas

Para concentrar a derivação no comando de repetição, usamos uma gramática reduzida. Os símbolos entre < > são não terminais; os demais representam terminais da linguagem.

```
<while_statement> ::= while <condition> do <compound_statement>
<condition> ::= <identifier> <= <integer_literal>
<compound_statement> ::= begin <statement_list> end
<statement_list> ::= <statement> ; <statement_list> | <statement>
<statement> ::= <if_statement> | <assignment_statement> | <procedure_statement>
<if_statement> ::= if <condition_if> then <statement> else <statement>
<condition_if> ::= <identifier> mod <integer_literal> = <integer_literal>
<assignment_statement> ::= <identifier> := <expression>
<expression> ::= <identifier> + <integer_literal>
<procedure_statement> ::= writeln ( <identifier> , <string_literal> )
<identifier> ::= i
<integer_literal> ::= 0 | 1 | 2 | 20
<string_literal> ::= 'eh par!' | 'eh impar!'
```

3. Derivação

A derivação abaixo começa no símbolo não terminal <while_statement> e substitui, passo a passo, cada não terminal até obter a sequência de terminais correspondente ao trecho desejado.

<while_statement>

⇒ while <condition> do <compound_statement>

⇒ while <identifier> <= <integer_literal> do <compound_statement>

⇒ while i <= <integer_literal> do <compound_statement>

⇒ while i <= 20 do <compound_statement>

⇒ while i <= 20 do begin <statement_list> end

⇒ while i <= 20 do begin <statement> ; <statement_list> end

⇒ while i <= 20 do begin <if_statement> ; <statement_list> end

⇒ while i <= 20 do begin <if_statement> ; <statement> end

⇒ while i <= 20 do begin <if_statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if <condition_if> then <statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if <identifier> mod <integer_literal> = <integer_literal> then <statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod <integer_literal> = <integer_literal> then <statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = <integer_literal> then <statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then <statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then <procedure_statement> else <statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then <procedure_statement> else <procedure_statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (<identifier> , <string_literal>) else <procedure_statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , <string_literal>) else <procedure_statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else <procedure_statement> ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (<identifier> , <string_literal>) ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , <string_literal>) ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , 'eh impar!') ; <assignment_statement> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , 'eh impar!') ; <identifier> := <expression> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , 'eh impar!') ; i := <expression> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , 'eh impar!') ; i := <identifier> + <integer_literal> end

⇒ while i <= 20 do begin if i mod 2 = 0 then writeln (i , 'eh par!') else writeln (i , 'eh impar!') ; i := i + <integer_literal> end

```
⇒ while i <= 20 do begin if i mod 2 = 0 then writeln ( i , 'eh par!' ) else writeln ( i , 'eh impar!' )  
; i := i + 1 end
```

Forma concreta em Pascal, com pontuação:

```
while i <= 20 do  
begin  
    if i mod 2 = 0 then  
        writeln(i, 'eh par!')  
    else  
        writeln(i, 'eh impar!');  
    i := i + 1;  
end;
```

4. Breve explicação textual

O comando while de Pascal é uma estrutura de repetição que executa um bloco de código enquanto uma condição especificada for verdadeira. Neste exemplo, o loop continua enquanto a variável 'i' for menor ou igual a 20. Dentro do loop, utilizamos uma estrutura condicional if...else para verificar se o valor atual de 'i' é par ou ímpar, exibindo a mensagem correspondente. Ao final de cada iteração, o valor de 'i' é incrementado. A derivação demonstra que este código não surge arbitrariamente: ele é uma sentença válida porque pode ser obtido por substituições sucessivas a partir das regras da gramática.

Referência: Free Pascal. Language Reference Guide, seções "While statement" e "If..then..else statement". Disponível em: <https://www.freepascal.org/docs-html/ref/ref.html>